Week5 Session- 5 Interview Questions

1. What is a Promise in Node.js, and how does it work?
A Promise is an object representing the eventual completion or failure of an asynchronous operation. It has three states:

- Pending: The initial state, operation not yet completed.
- Fulfilled: The operation completed successfully, resolve() is called.
- Rejected: The operation failed, reject() is called.
- Promises simplify handling asynchronous operations by avoiding callback hell.

2. What is the purpose of the async keyword in Node.js?
The async keyword defines a function that always returns a Promise. Inside the function, you can use await to pause execution until a Promise is resolved.

3. Can await be used outside of an async function? Why or why not?
No, await can only be used inside an async function because it pauses the execution, which requires the function to return a Promise.

4. What is the difference between .then() and await?
.then() : Used for chaining promises. It takes two callbacks (onFulfilled, onRejected).
await: Pauses execution in async functions, allowing you to write sequential-style code.

5. What happens if a Promise is neither resolved nor rejected?
If a Promise remains in a pending state, it results in a memory leak as the promise never settles. Always ensure every Promise either resolves or rejects.

6. What is the benefit of using Promise.all?
Promise.all executes multiple Promises concurrently and resolves only when all Promises are resolved. It improves performance for parallel operations.

7. What is a promise in Node.js, and how does it solve callback hell?
A promise is an object that represents the eventual result (resolved or rejected) of an asynchronous operation. Promises help flatten nested callbacks and improve readability.

8. What are some limitations of async/await?
Limitations are:

- It is sequential by default, so it may not be efficient for parallel tasks.
- It cannot be used outside of async functions.
- Debugging can be challenging without proper error handling.

9. Explain the lifecycle of a Promise in Node.js.
A Promise goes through three stages:

- **Pending**: The initial state where the Promise neither resolves nor rejects.
- **Fulfilled**: When the asynchronous operation completes successfully, and resolve is called.
- **Rejected**: When the operation fails, and reject is called.

These states allow better management of asynchronous operations and error handling.